



**VietNam National University
University of Engineering and Technology**

EMBEDDED SYSTEM FUNDAMENTALS

(ELT3240, NHẬP MÔN HỆ THỐNG NHÚNG)

Dr. Nguyễn Kiêm Hùng

Email: kiemhung@vnu.edu.vn

Laboratory for Smart Integrated Systems



Introduction to
Embedded Systems

Week
1-2

Introduction to C
Language

Week
3

CPU:
ARM Cortex-M

Week
4

Memory
and Interfaces

Week
5-6

ARM-based
Embedded System

Week
7

You Are
Here.

Embedded Software

Week
8-9

Real-time
Operating systems

Week
10-12

Interfacing Embedded
With Real-World

Week
13-14

Project

Week
15

Laboratory for Smart Integrated Systems

Curriculum Path

Objectives

In this lecture you will be introduced to:

- Definition of the embedded software
- Some useful structures for embedded software development.
- Models of programs, such as data flow and control flow graphs.
- An introduction to compilation methods.
- Analyzing and optimizing programs for performance, size, and power consumption.

Outline



- **Embedded Software**

- Embedded Software components
- Representations of programs
- Assembly and linking
- Compilation flow
- Summary

Embedded Software

Definition

“Embedded software is ***computer software***, written to control machines or devices that are not typically thought of as computers. It is typically specialized for the particular hardware that it runs on and has ***time and memory constraints***. This term is sometimes used interchangeably with ***firmware***.”

“Firmware” means that:

- The embedded software only repeatedly carries out particular designated functions,
- The software is stored in ROM and executes on reset.

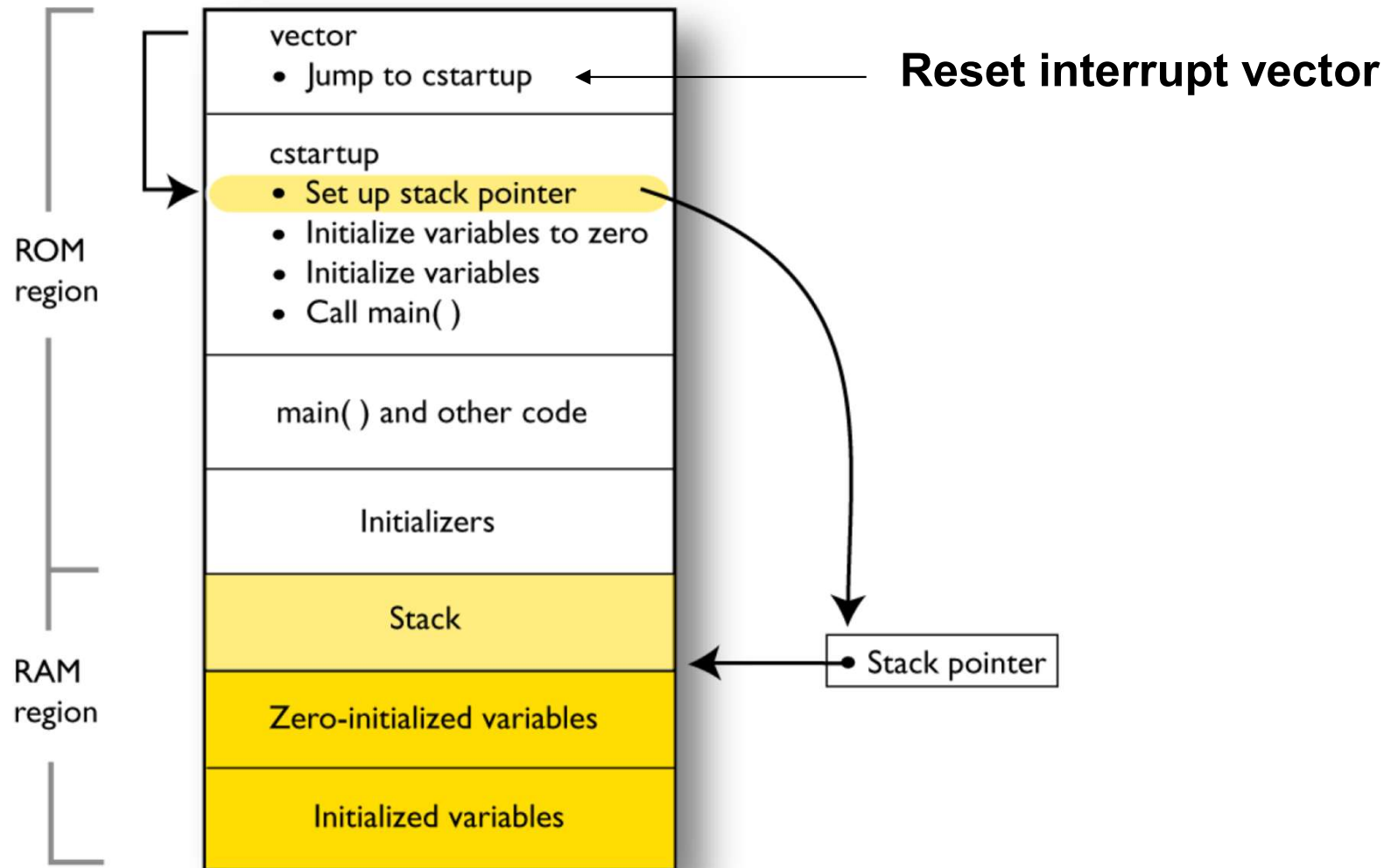
Embedded Software Overview

Common Components:

- Interrupt Vector Table
 - Reset vector
- Startup code:
 - initialize the system—both the hardware (e.g. stack pointer) and the software system (e.g. global variables)
- Application code
- Libraries
- Interrupt/Exception Handler

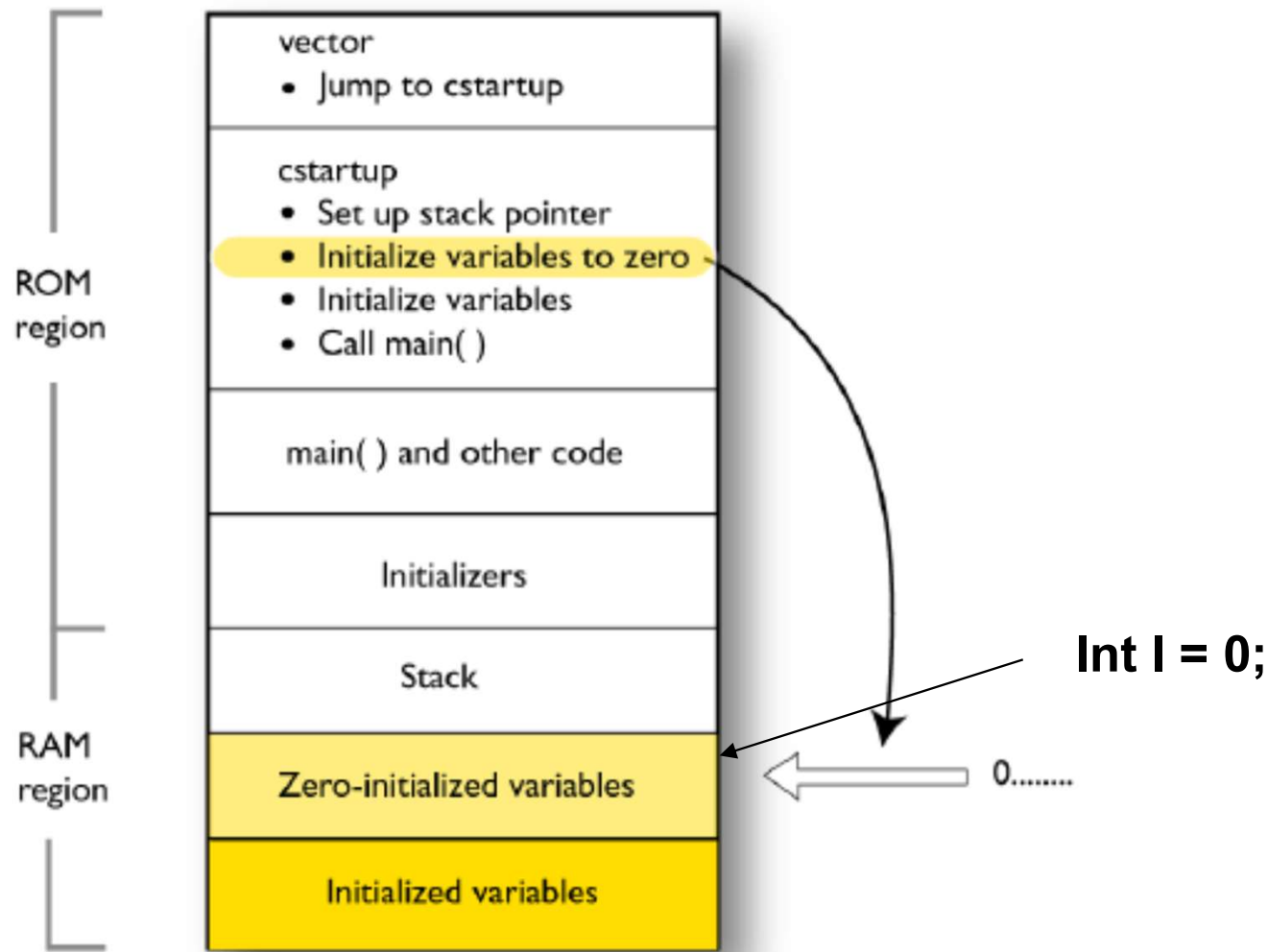
Embedded Software Overview

How does a firmware execute:



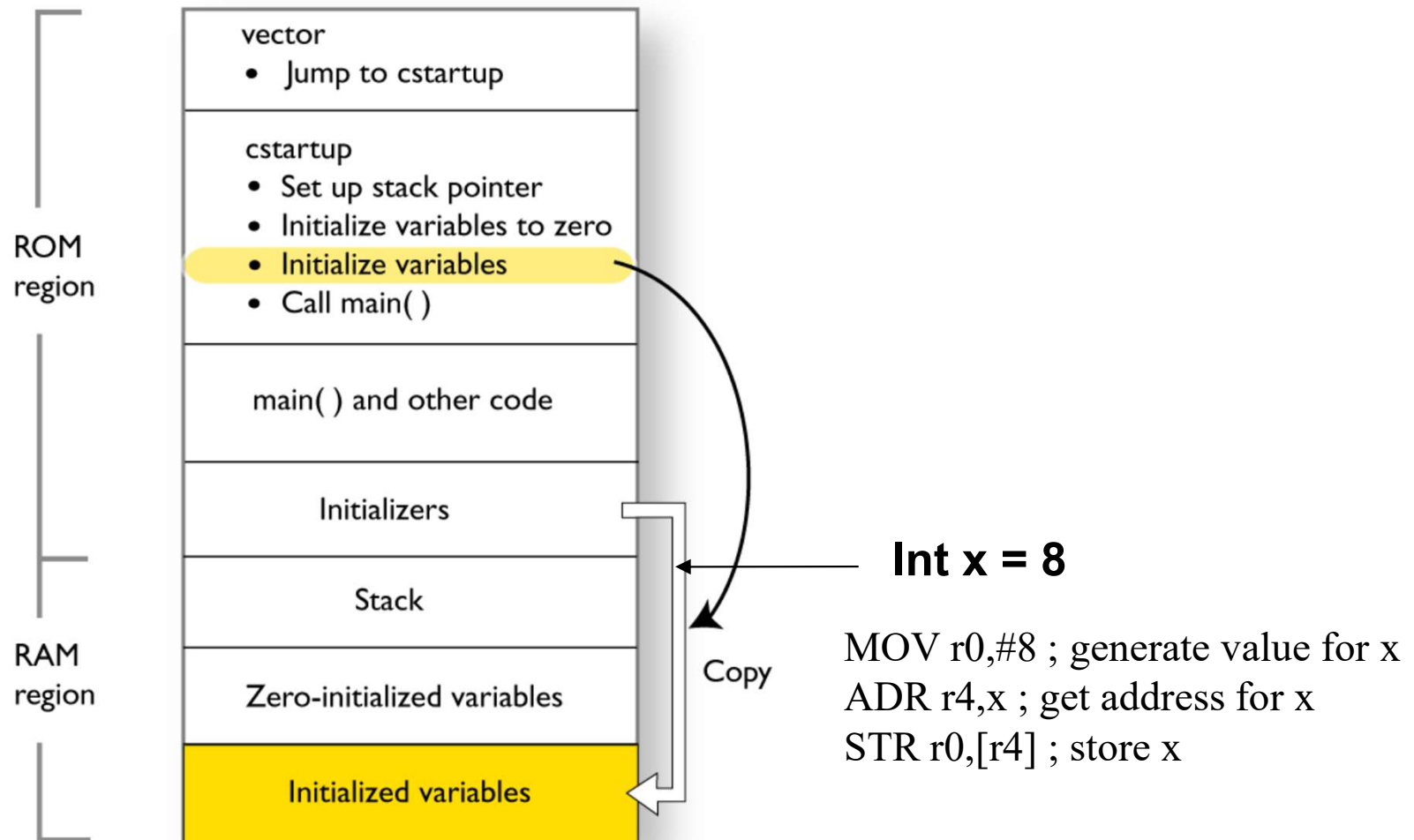
Embedded Software Overview

How does a firmware execute:



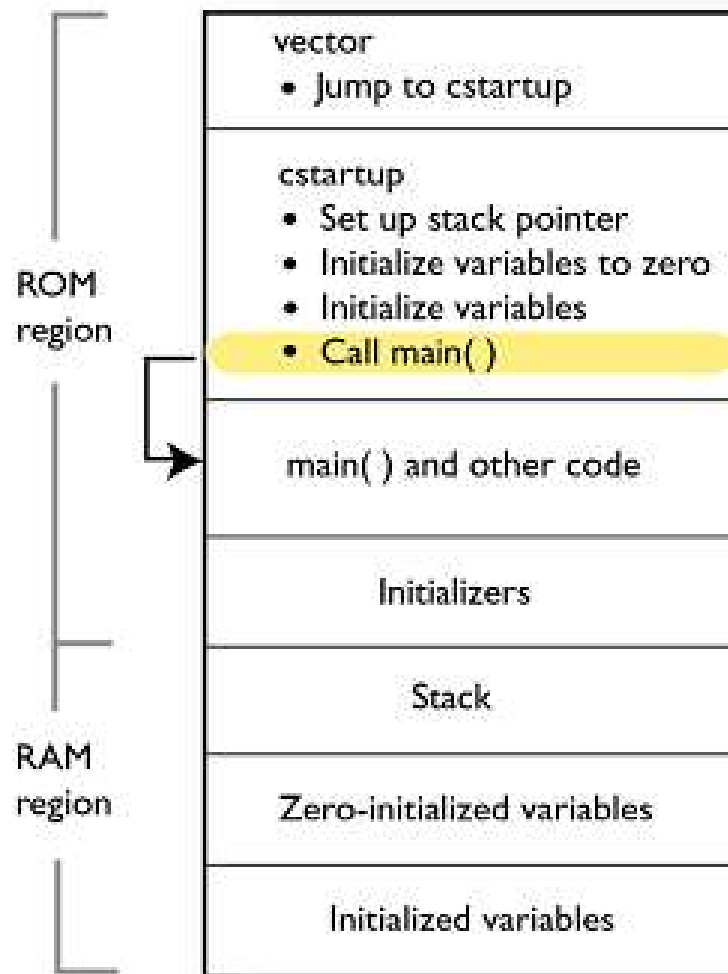
Embedded Software Overview

How does a firmware execute:



Embedded Software Overview

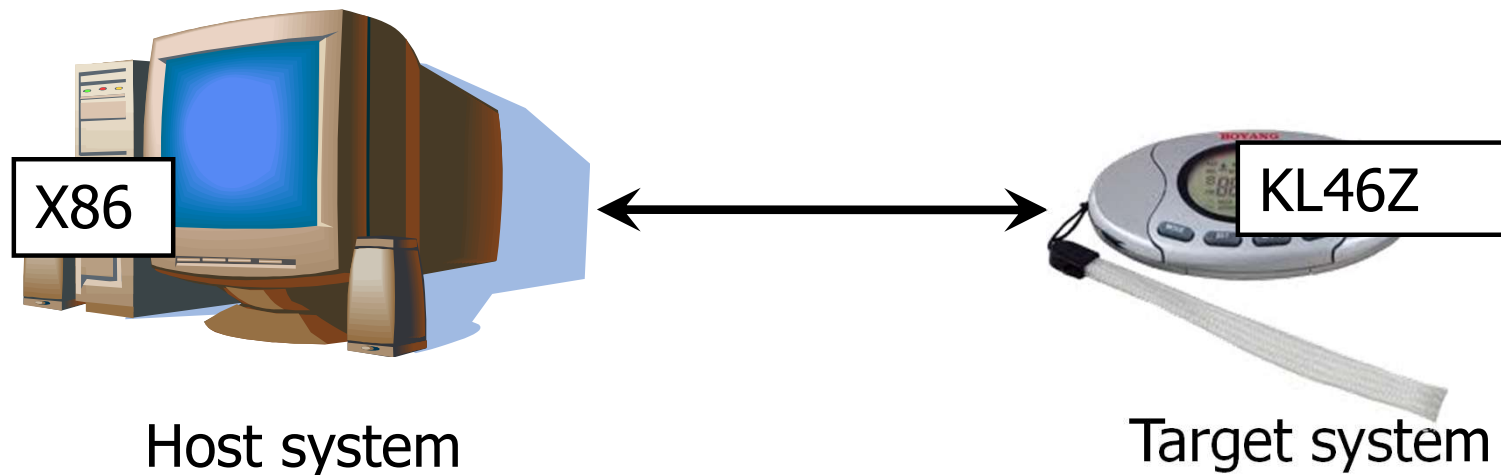
How does a firmware execute:



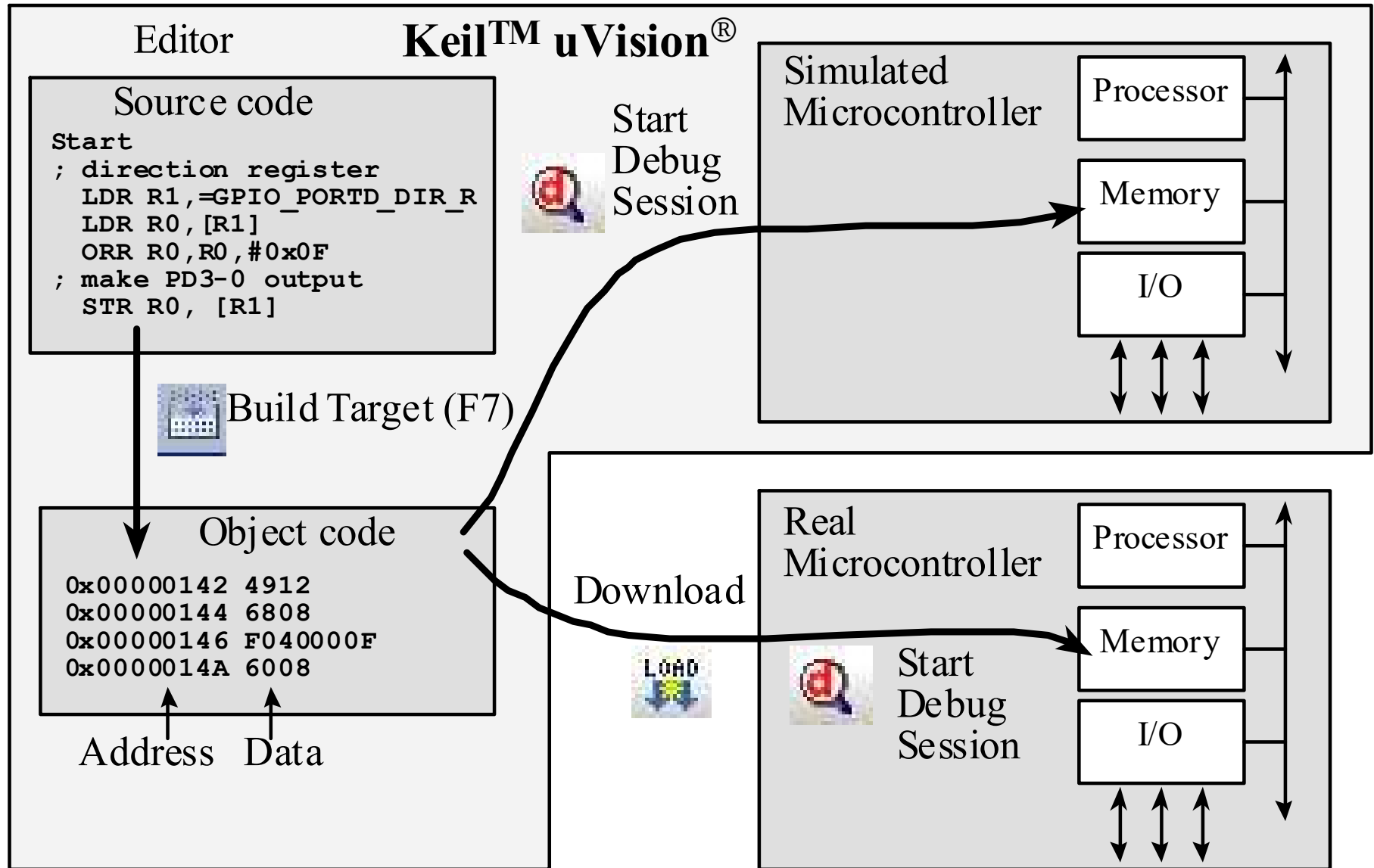
Main() is typically implemented as an endless loop which is either interrupt-driven or uses polling for controlling external interaction or internal events!

Development Environment

- **Host:** a computer running programming tools for development
- **Target:** the HW on which code will run
- After program is written, compiled, assembled and linked, it is transferred to the target

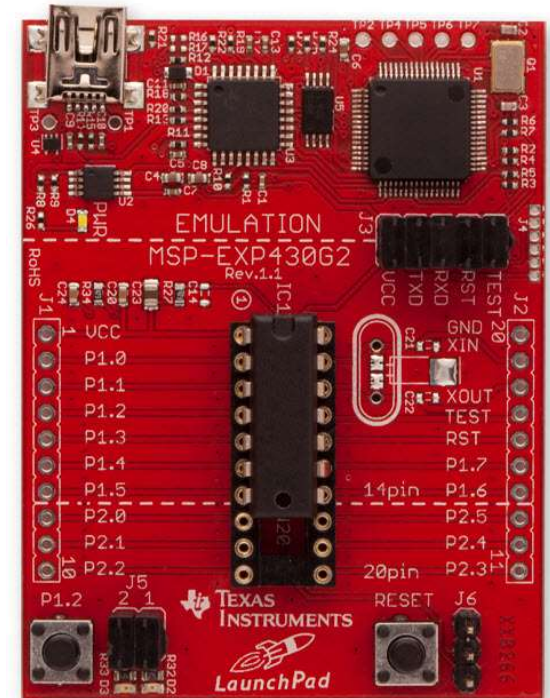


SW Development Environment



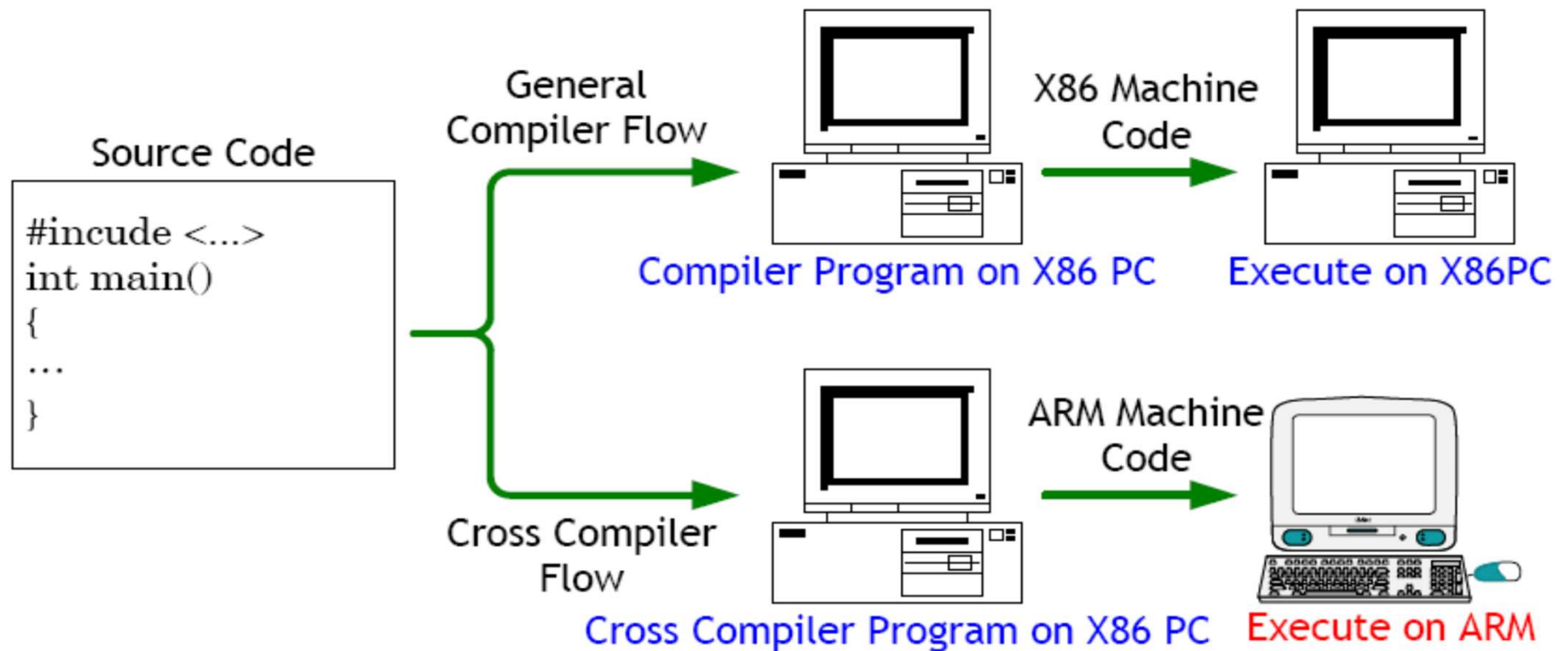
What If Real HW Not Available?

- Development board:
 - Before real hardware is built, software can be developed and tested using development boards
 - Development boards usually have the same CPU as the end product and provide many IO peripherals for the developed software to use as if it were running on the real end product
- Tools for program development
 - Integrated Development Environment (IDE): cross compiler, linker, loader, ...
 - OS and related libraries and packages



Cross Compiler

- Runs on host but generates code for target
 - Target usually have different architecture from host. Hence compiler on host has to produce binary instructions that will be understood by target



Outline



- Embedded Software
- **Embedded Software components**
- Representations of programs
- Assembly and linking
- Compilation flow
- Summary

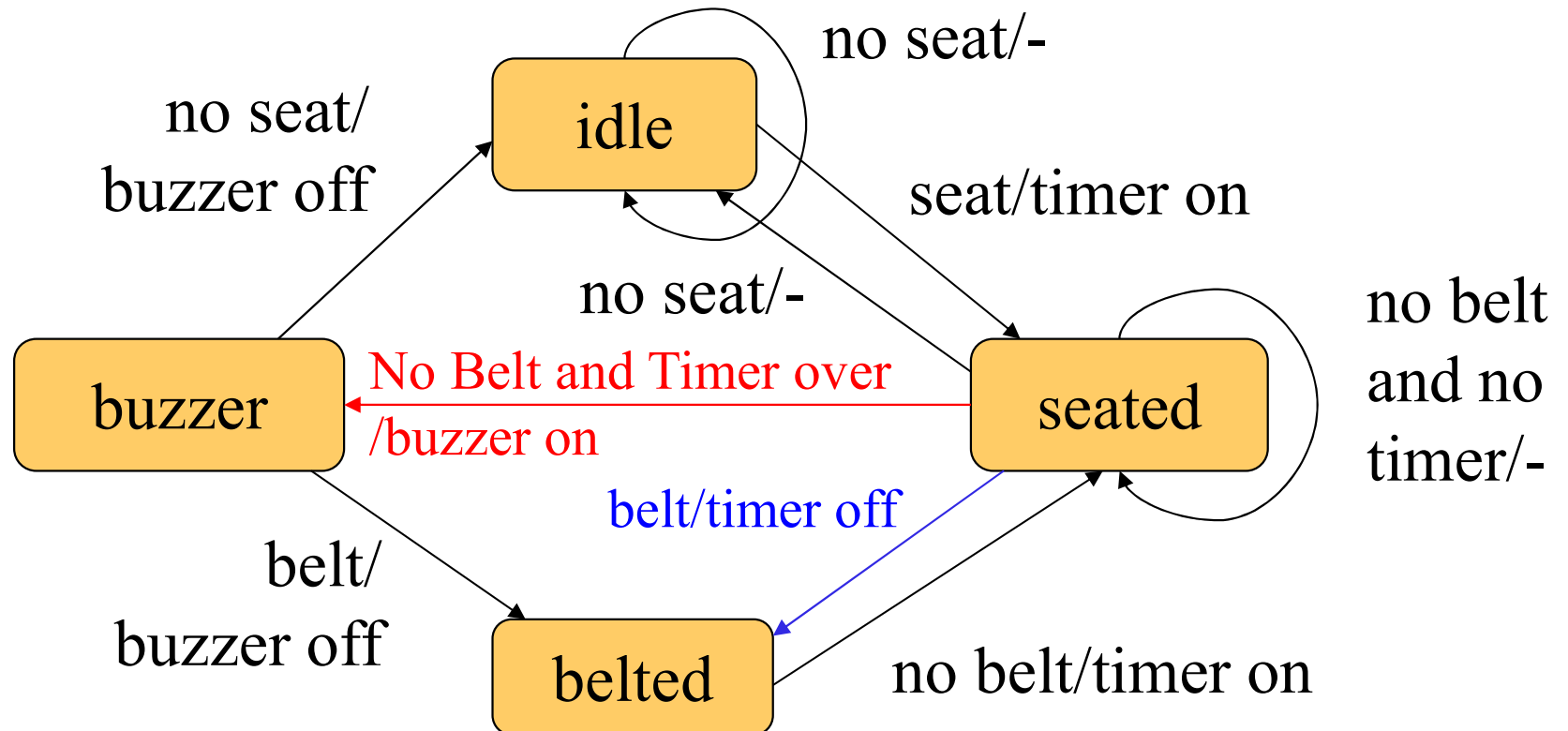
Software state machine

- State machine keeps internal state as a variable, **changes state based on inputs and the current state of the system.**
- Uses for:
 - **control-dominated code;**
 - **reactive systems:** inputs appear intermittently rather than as periodic samples

State machine example

Seat belt controller:

- Two inputs: seat sensor and belt sensor
- One Timer:
- One Output: Buzzer

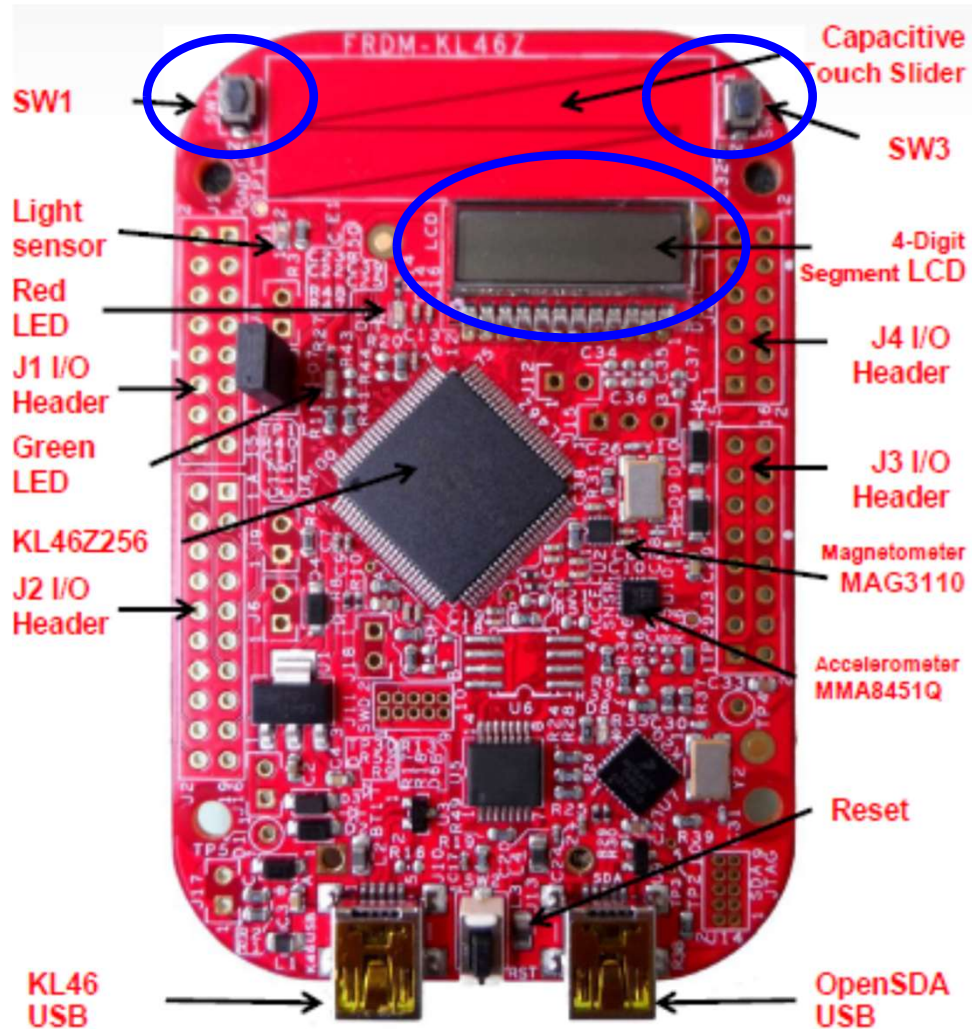


State machine example

```
#define IDLE 0
#define SEATED 1
#define BELTED 2
#define BUZZER 3
Switch (state) { /* check the current state */
    case IDLE:
        if (seat){ state = SEATED; timer_on = TRUE; }
        /* default case is self-loop */
        break;
    case SEATED:
        if (belt) state = BELTED; /* won't hear the buzzer */
        else if (timer_over) state = BUZZER; /* didn't put on belt in time */
        /* default case is self-loop */
        break;
    case BELTED:
        if (!seat) state = IDLE; /* person left */
        else if (!belt) state = SEATED; /* person still in seat */
        break;
    case BUZZER:
        if (belt) state = BELTED; /* belt is on---turn off buzzer */
        else if (!seat) state = IDLE; /* no one in seat--turn off buzzer */
        break;
}
```

Project

Seat belt controller



- Hardware Requirement:
 - 2 user LEDs
 - 2 user push buttons
 - Timer
 - Option: 4-digit 7-segment LCD module

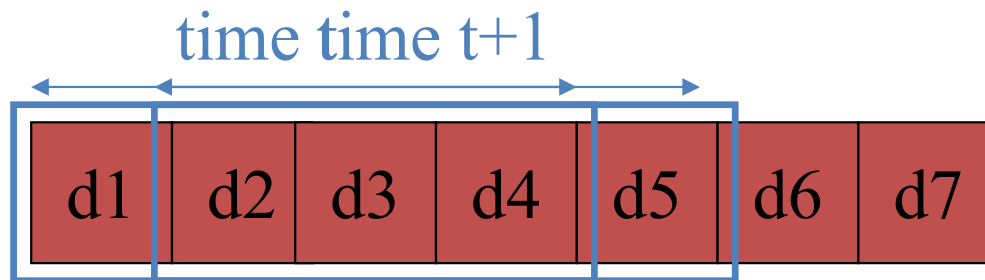
Project

Seat belt controller

- Description: The controller turns on a buzzer if a person sits in a seat and does not fasten the seat belt within a fixed amount of time.
- This system has two inputs and two outputs.
 - SW1 represents the sensor that detects when a person has sat down,
 - SW2 represents the seat belt sensor that tells when the belt is fastened,
 - Red LED is the output that represents the buzzer.
 - Blue LED is the output that represents the belted state.
 - LCD (option): displays current state of the system
- A timer is for setting the required time interval before turn on buzzer

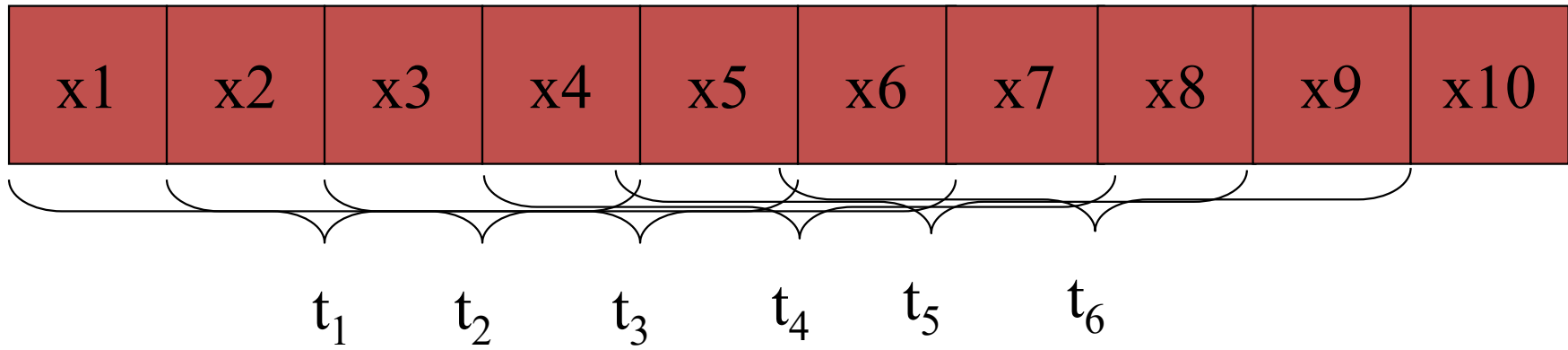
Signal processing and circular buffer

- Commonly used in signal processing:
 - new data constantly arrives and must be processed in real time;
 - each datum has a limited lifetime.

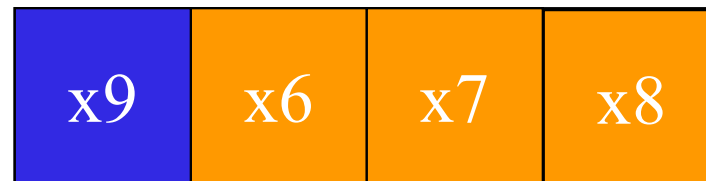


- In an embedded system we must not only emit outputs in real time, but we must also do that using a minimum amount of memory
 - Use a circular buffer to hold the data stream.

Circular buffer



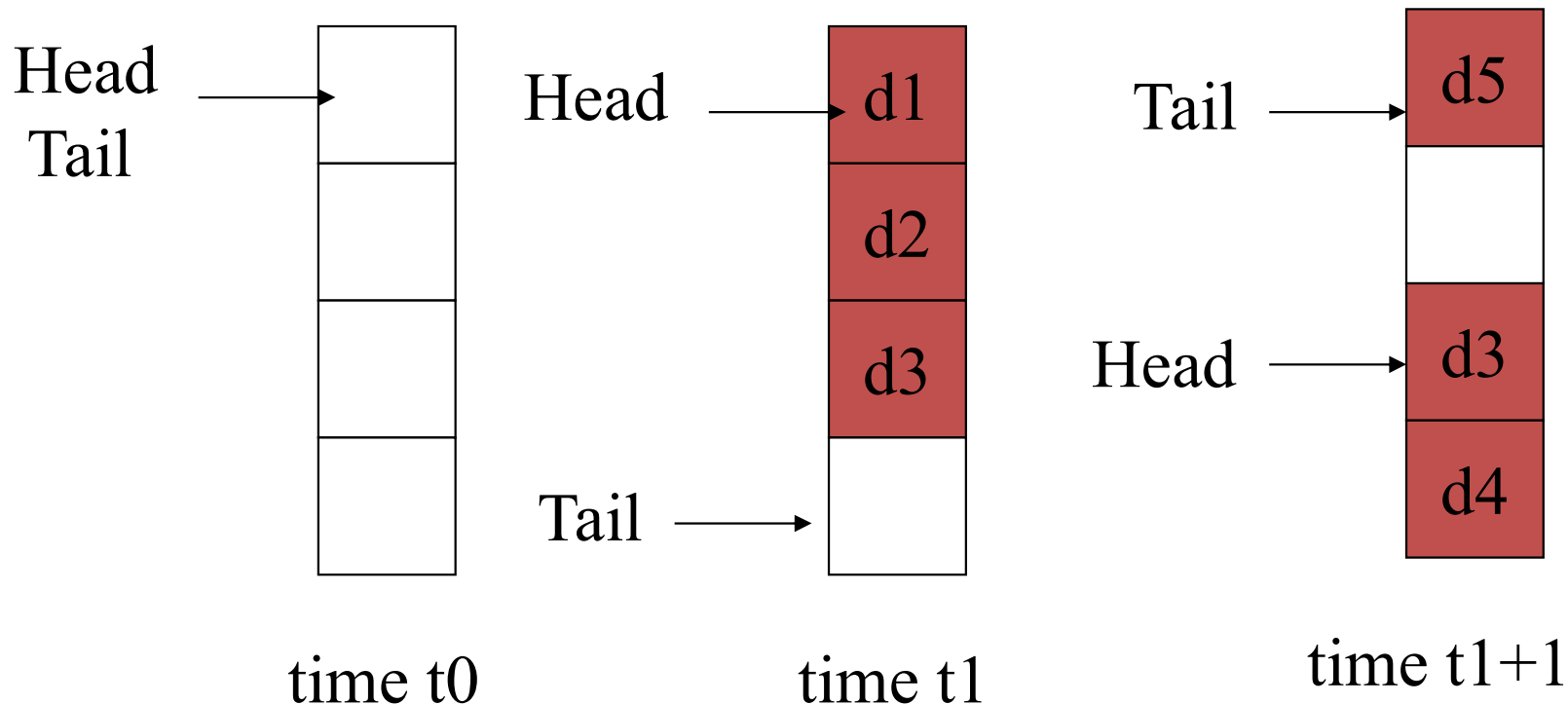
Data stream



Circular buffer

Queues

- Are used whenever data may arrive and depart at somewhat unpredictable times
- Implementation:
 - Declare the array
 - The variables head and tail keep track of the two ends of the queue
 - Two error conditions: Reading from an empty queue and writing to a full queue.



Outline

- Embedded Software
- Software components
- **Representations of programs**
- Assembly and linking
- Compilation flow
- Summary



Models of programs

- Source code is not a good representation for programs:
 - many different types of source code,
 - Clumsy.
- Require a single model:
 - to describe all of types of source code,
 - and to perform many useful analyses on the model more easily
- Compilers derive intermediate representations to manipulate and optimize the program.

Data flow graph

- **DFG**: data flow graph.
- Models a **program (also called basic block)**:
 - **with only one entry and exit point.**
 - **with only data operations (e.g. arithmetic and other computations) and no control operations (conditionals).**

Data flow graph

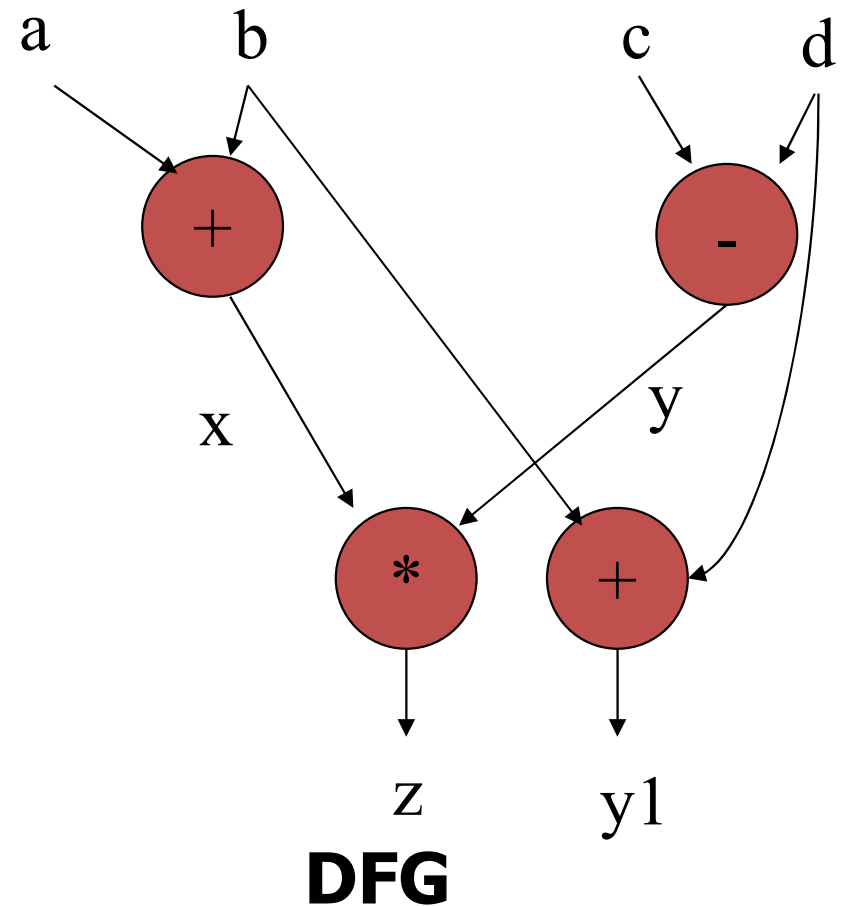
$x = a + b;$

$y = c - d;$

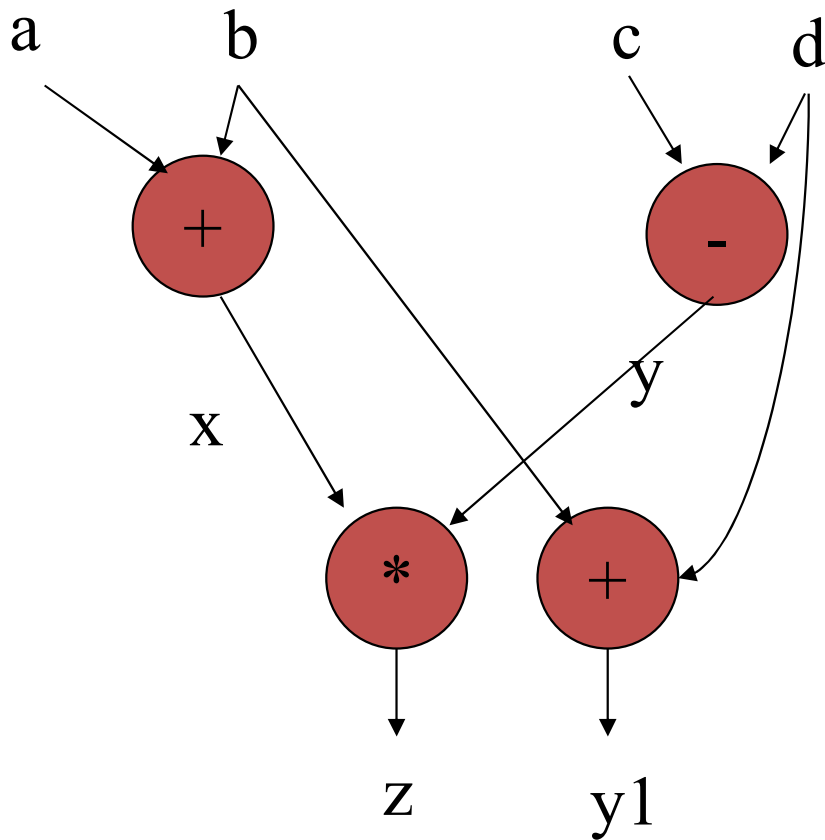
$z = x * y;$

$y1 = b + d;$

A basic block in single-assignment form



DFGs and partial orders



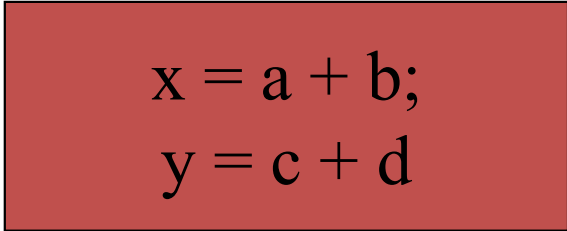
- DFG shows the order in which the operations are performed in the program
- Partial order:
 - $[a+b, c-d]; [b+d, x*y]$
- Can do pairs of operations in any order.
- DFG help us to determine feasible reorderings of the operations, so to reduce pipeline or cache conflicts

Control-data flow graph

- **CDFG: model both data operations** (arithmetic and other computations) **and control operations (conditionals).**
- Uses data flow graphs as components, adding constructs to describe control.
- Two types of nodes:
 - **Decision node:** describe all the types of control in a sequential program;
 - **Data flow node:** represent a basic block.

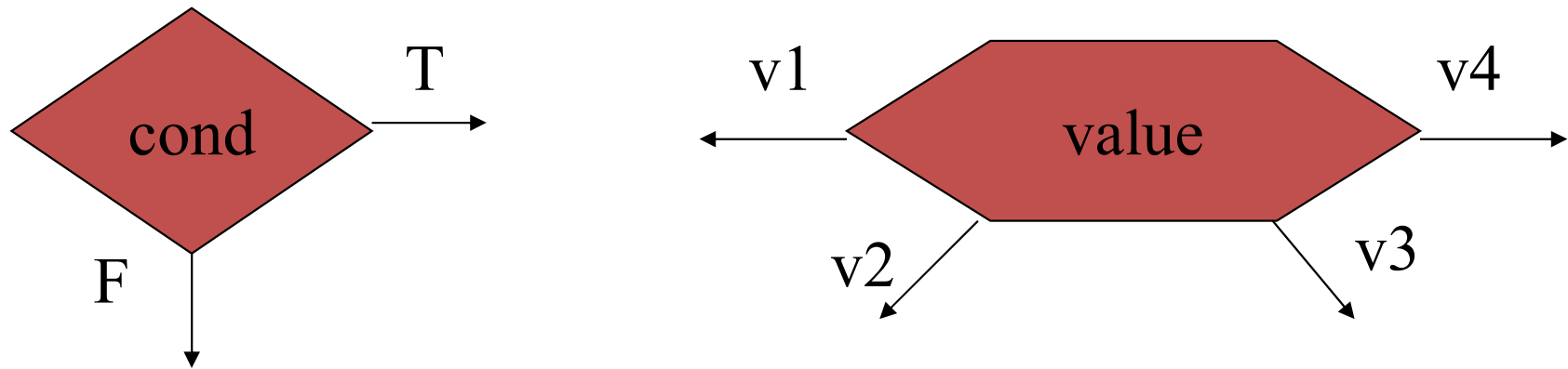
Data flow node

Encapsulates a data flow graph to represent basic block form:



```
x = a + b;  
y = c + d
```

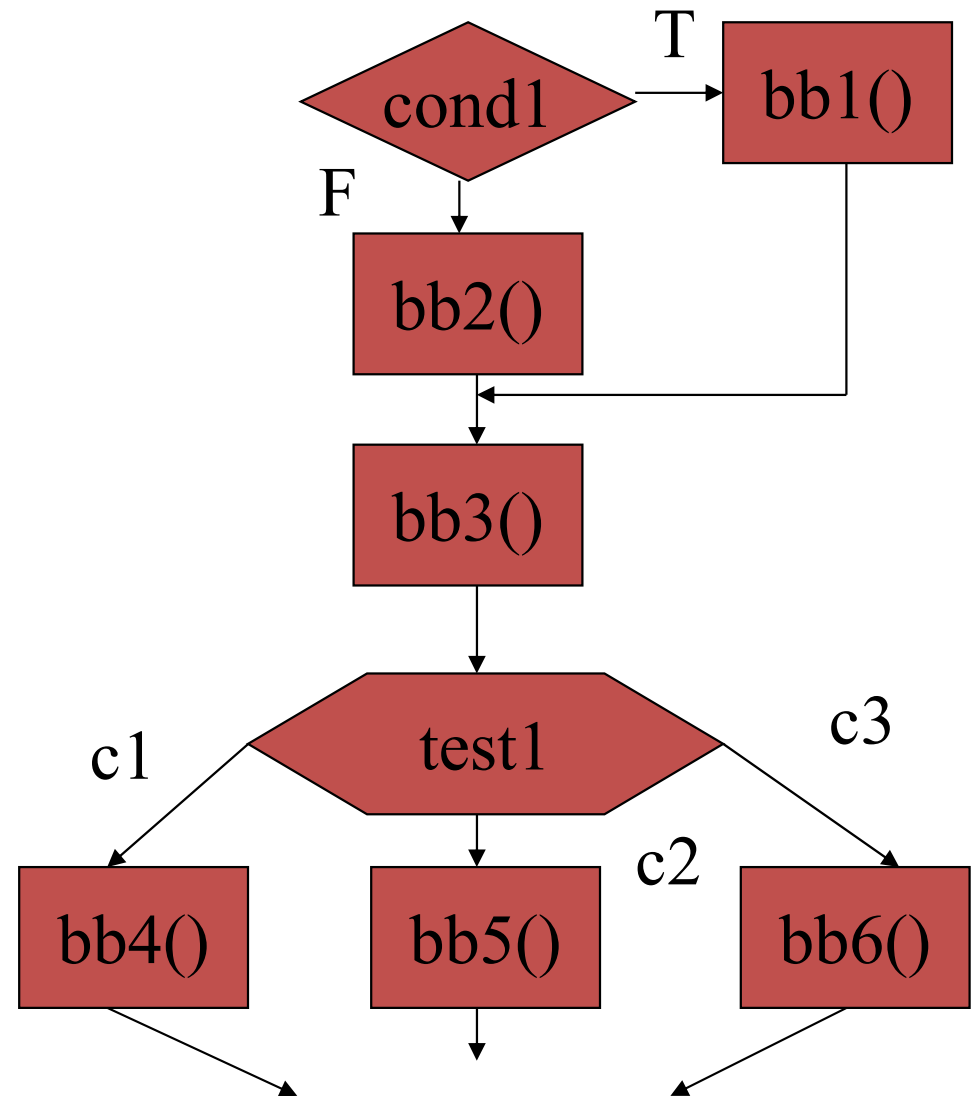
Control Nodes



Equivalent forms

CDFG example

```
if (cond1) bb1();  
else bb2();  
bb3();  
switch (test1) {  
    case c1: bb4(); break;  
    case c2: bb5(); break;  
    case c3: bb6(); break;  
}
```

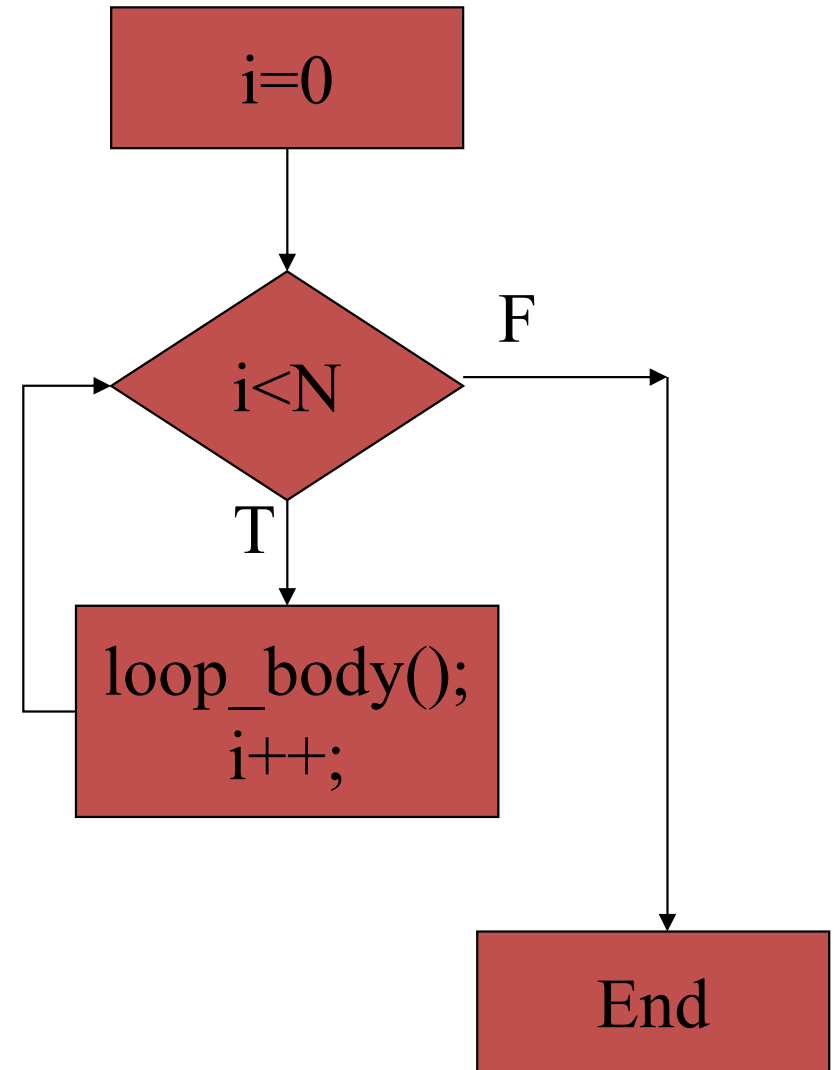


for loop

```
for (i=0; i<N; i++)  
    loop_body();
```

for loop

```
i=0;  
while (i<N) {  
    loop_body(); i++; }  
  
equivalent
```

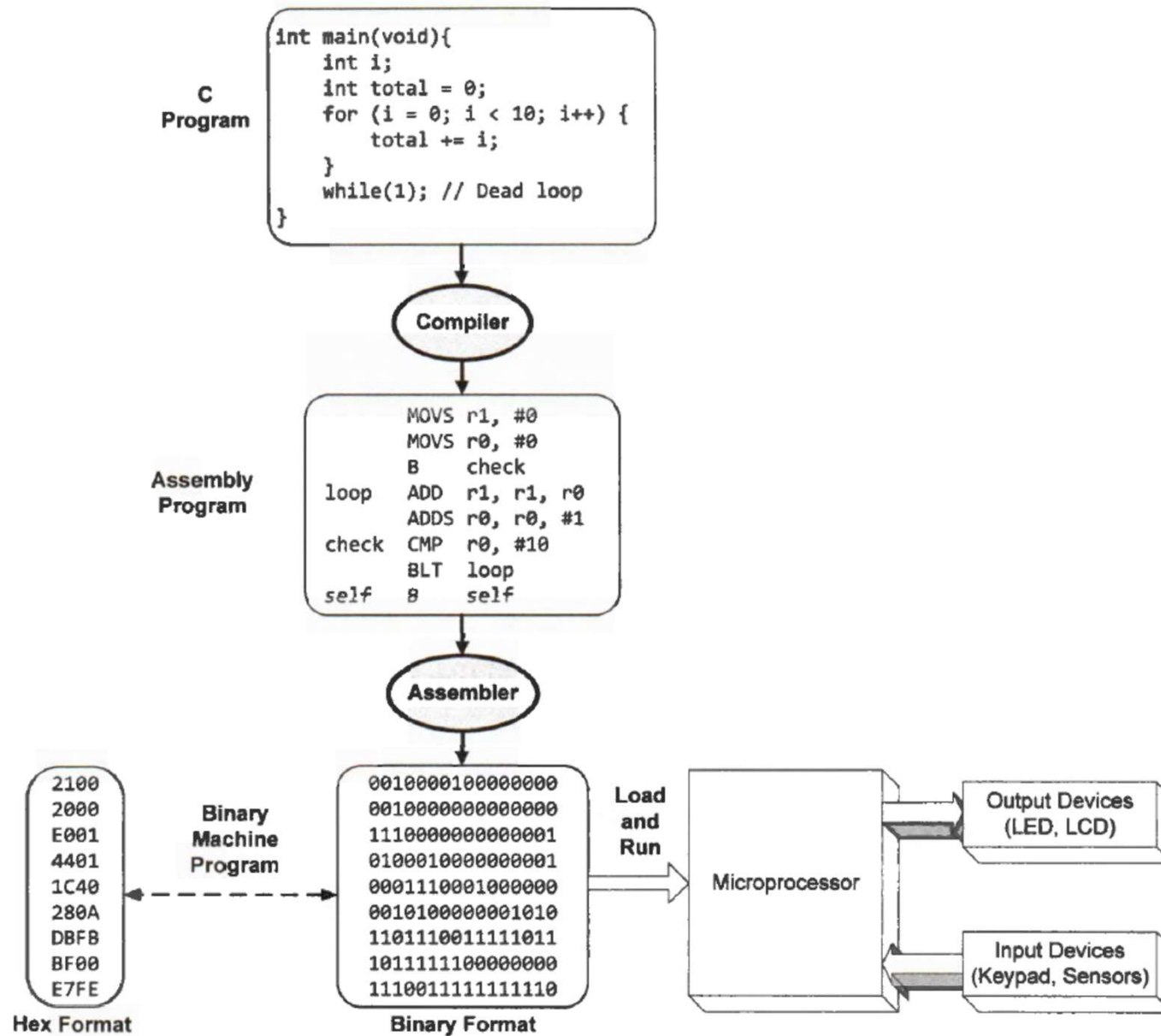


Outline (h1h2)

- Embedded Software
- Software components
- Representations of programs
- **Assembly and linking**
- Compilation flow
- Summary



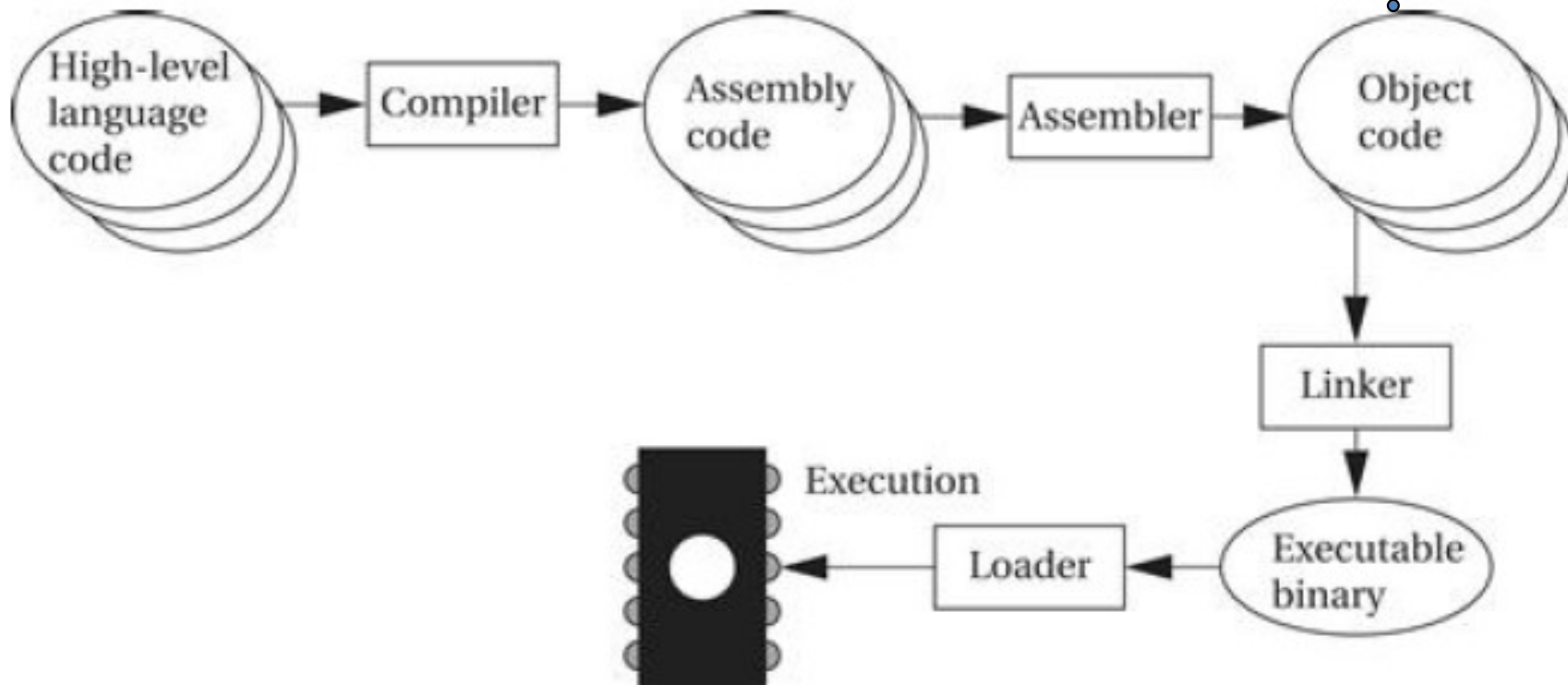
Assembly and linking



Assembly and linking

- Last steps in compilation:

Describes the instructions and data in binary format.



Multiple-module programs

- Programs may be composed from several files.
- Addresses become more specific during processing:
 - **relative addresses** are measured relative to the start of a module;
 - **absolute addresses** are measured relative to the start of the CPU address space.

Assemblers

- Major tasks:
 - generate binary representation for symbolic assembly instructions;
 - translate labels into addresses;
 - handle pseudo-ops (data, etc.).
- Generally one-to-one translation.
- Assembly labels: represent locations of instructions and data

ORG 100 ; pseudo-ops

label1 ADR r4,c ;

Two-pass assembly

- Pass 1:
 - generate symbol table:
 - determining the address of each label
- Pass 2:
 - generate binary instructions
 - using the label values computed in the first pass

Symbol table

ORG 0x7

ADD r0,r1,r2

xx 0xB

xx ADD r3,r4,r5

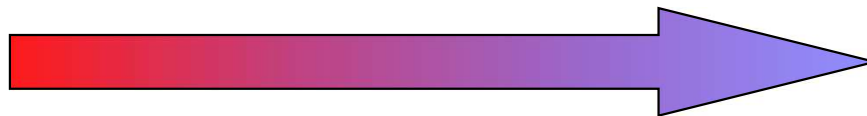
yy 0x13

CMP r0,r3

yy SUB r5,r6,r7

assembly code

symbol table



First Pass

Symbol table generation

- Use program location counter (**PLC**) to determine address of each location.
- Scan program, keeping count of PLC.
- Addresses are generated at assembly time, not execution time.

Symbol table example

ORG 0x7

	PLC=0x7	ADD r0,r1,r2	xx	0xB
xx	PLC=0xB	ADD r3,r4,r5	yy	0x13
	PLC=0xF	CMP r0,r3		
yy	PLC=0x13	SUB r5,r6,r7		



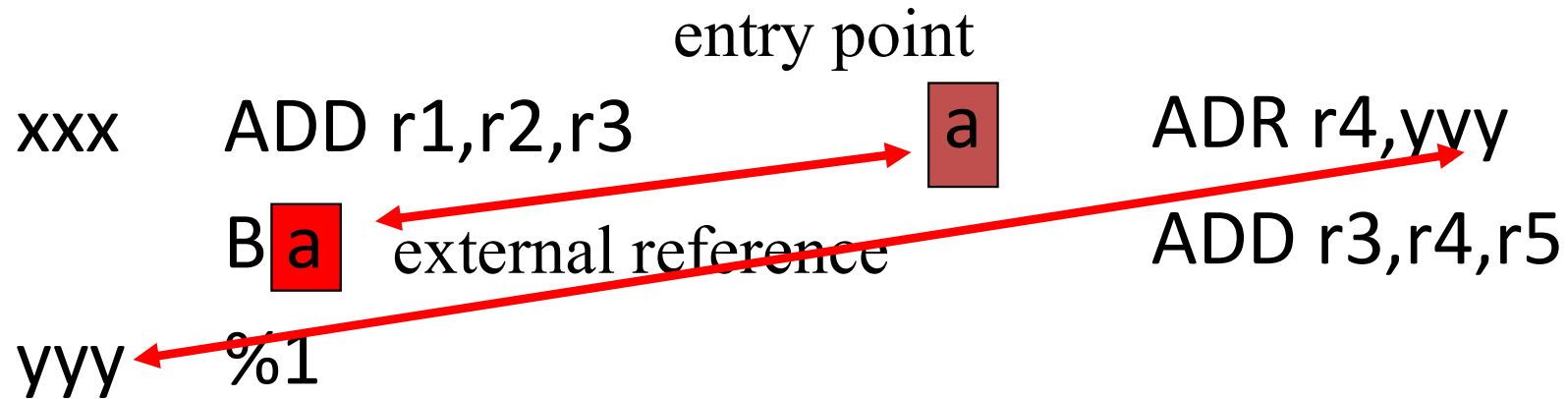
Relative address generation

- Some label values may not be known at assembly time.
- Labels within the module may be kept in relative form.
- Must keep track of external labels---can't generate full binary for instructions that use external labels.

Linking

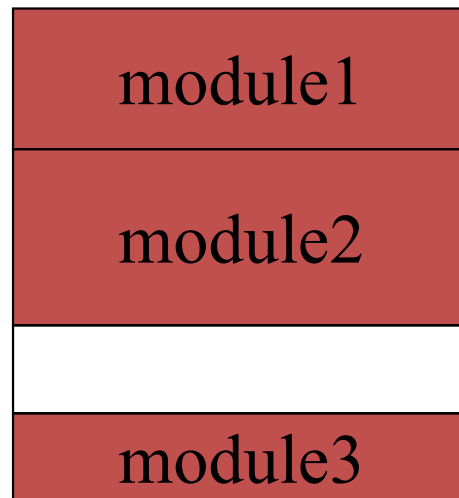
- **Combines several object modules into a single executable module.**
 - An assembly language program are usually written as several smaller pieces + uses library routines rather than as a single large file.
- **Jobs:**
 - put modules in order;
 - resolve labels across modules.

External References and entry points



Module ordering

- Code modules must be placed in absolute positions in the memory space.
- **Load map** or linker flags control the order of modules.
 - Some data structures and instructions must be put at precise memory locations
 - Many different types of memory may be installed at different address ranges.



Outline

- Embedded Software
- Software components.
- Representations of programs.
- Assembly and linking.
- **Compilation flow.**
- Summary



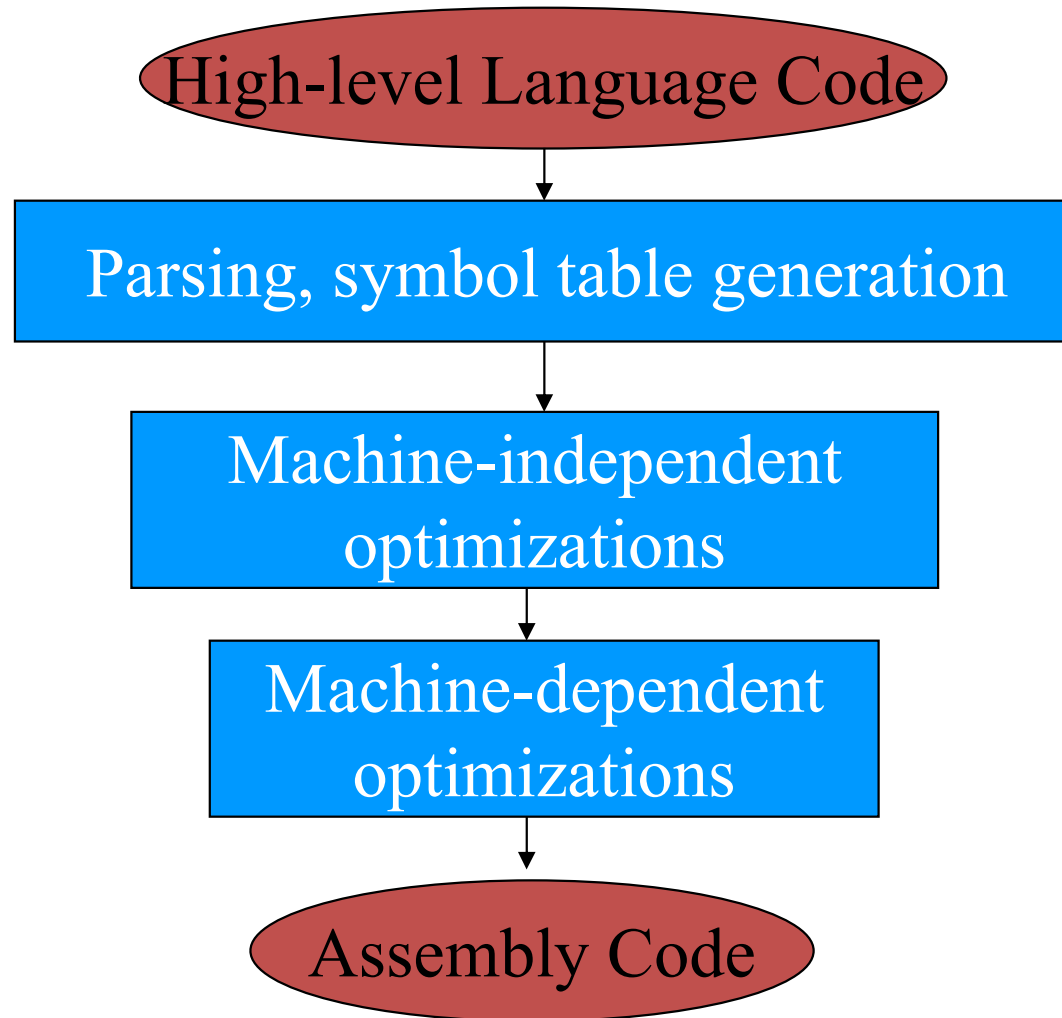
Compilation

- Why need to understand Compilation process
 - how a high-level language program is translated into instructions: interrupt handling instructions, placement of data and instructions in memory, etc
 - how code is generated can help you meet your **performance goals**:
 - either by writing high-level code that gets compiled into the instructions you want
 - or by recognizing when you must write your own assembly code.

Compilation

- Compilation process:
 - compilation = translation + optimization
 - The high-level language program is translated into the lower-level form of instructions;
 - optimizations try to generate better instruction sequences
- Compiler determines quality of code:
 - use of CPU resources;
 - memory access scheduling;
 - code size.

Basic compilation phases



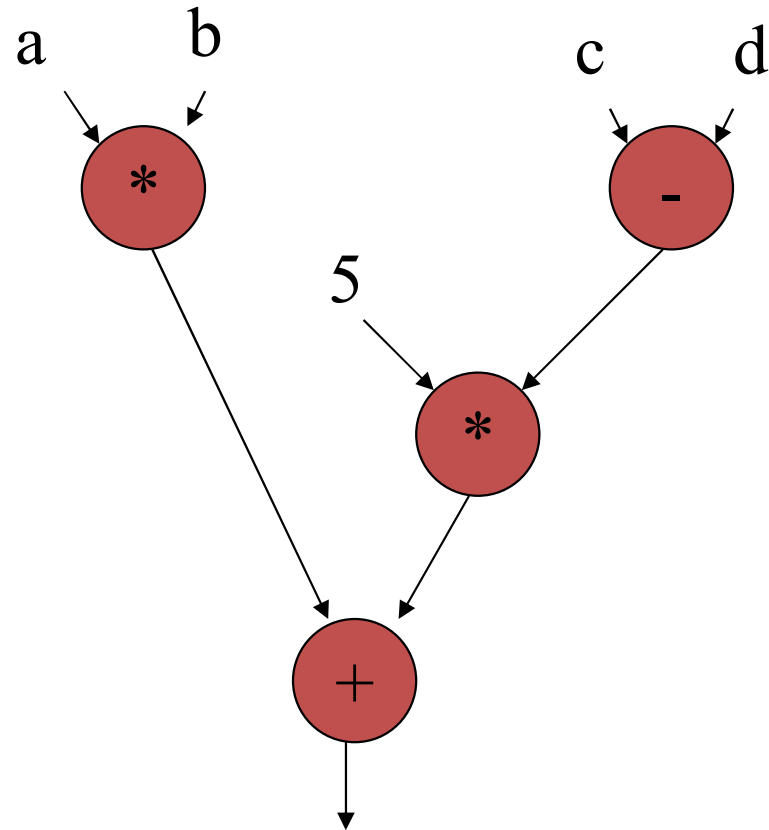
Statement translation and optimization

- Source code is translated into intermediate form such as CDFG.
- CDFG is transformed/optimized.
- CDFG is translated into instructions with optimization decisions.
- Instructions are further optimized.

Arithmetic expressions

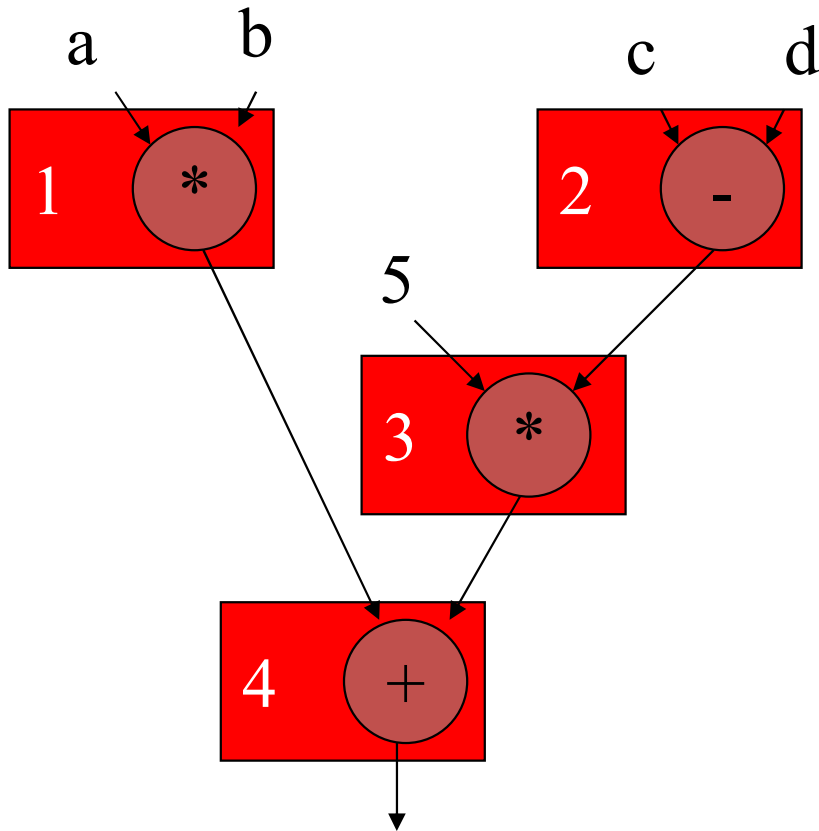
$$X = a * b + 5 * (c - d)$$

expression



DFG

Arithmetic expressions, cont'd.



DFG

```
ADR r4,a
MOV r1,[r4]
ADR r4,b
MOV r2,[r4]
MUL r3,r1,r2
```

```
ADR r4,c
MOV r1,[r4]
ADR r4,d
MOV r5,[r4]
SUB r6,r4,r5
MUL r7,r6,#5
ADD r8,r7,r3
ADR r1,x
STR r8,[r1]
```

Assembly code

Compiled code for arithmetic expressions

```
ldr r2, [fp, #-16]
ldr r3, [fp, #-20]
mul r1, r3, r2 ; multiply
ldr r2, [fp, #-24]
ldr r3, [fp, #-28]
rsb r2, r3, r2 ; subtract
```

```
mov r3, r2
mov r3, r3, asl #2
add r3, r3, r2 ; add
add r3, r1, r3 ; add
str r3, [fp, #-32] ; assign
```

code generated by the ARM gcc compiler

Control code generation

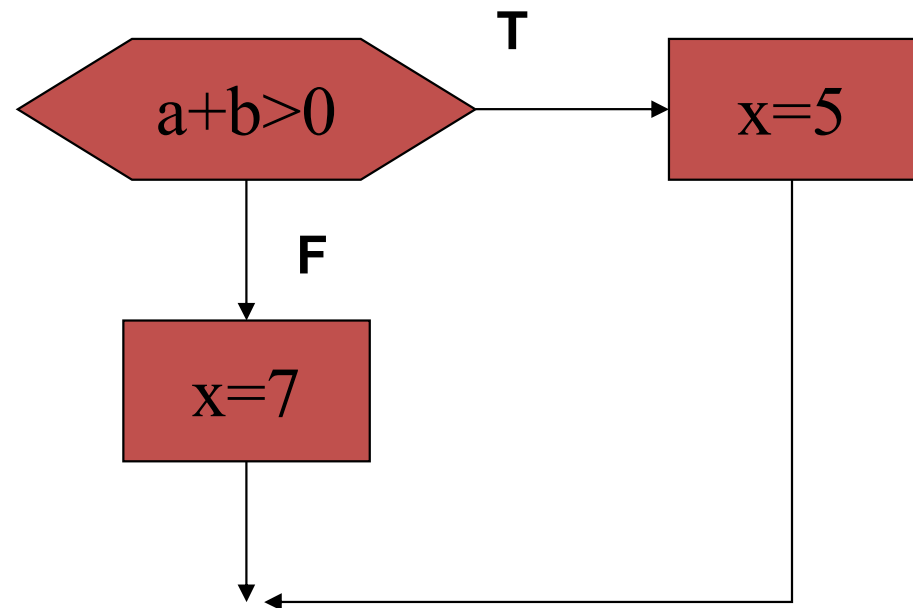
if ($a+b > 0$)

$x = 5;$

else

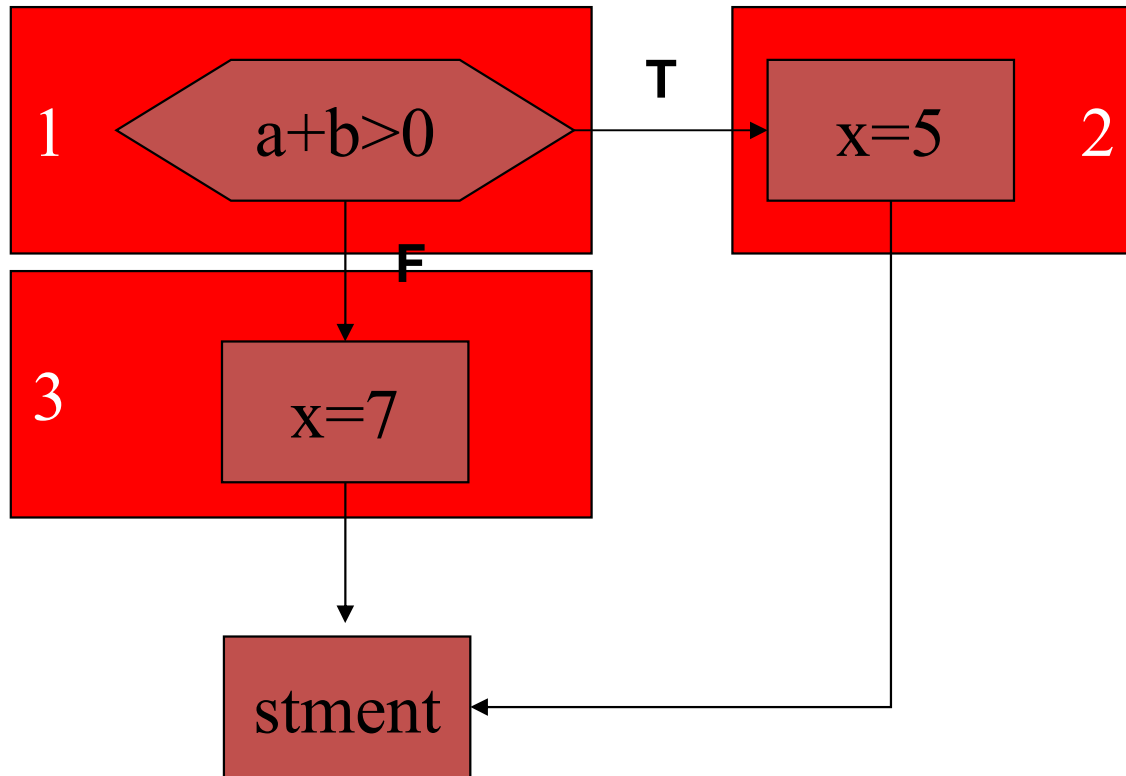
$x = 7;$

expression



DFG

Control code generation, cont'd.



```
ADR r5,a
LDR r1,[r5]
ADR r5,b
LDR r2,[r5]
ADD r3,r1,r2
CMP R3, #0
BLE label3
LDR r3,#5
ADR r5,x
STR r3,[r5]
```

B stmtent

```
Label3
LDR r3,#7
ADR r5,x
STR r3,[r5]
stmtent ...
```

Compiled code for control

```
ldr r2, [fp, #-16]
ldr r3, [fp, #-20]
add r3, r2, r3
cmp r3, #0 ; test the
    branch condition
ble .L3 ; branch to false
    block if <=
```

```
mov r3, #5 ; true block
str r3, [fp, #-32]
b .L4 ; go to end of if
    statement
.L3: ; false block
mov r3, #7
str r3, [fp, #-32]
.L4:
```

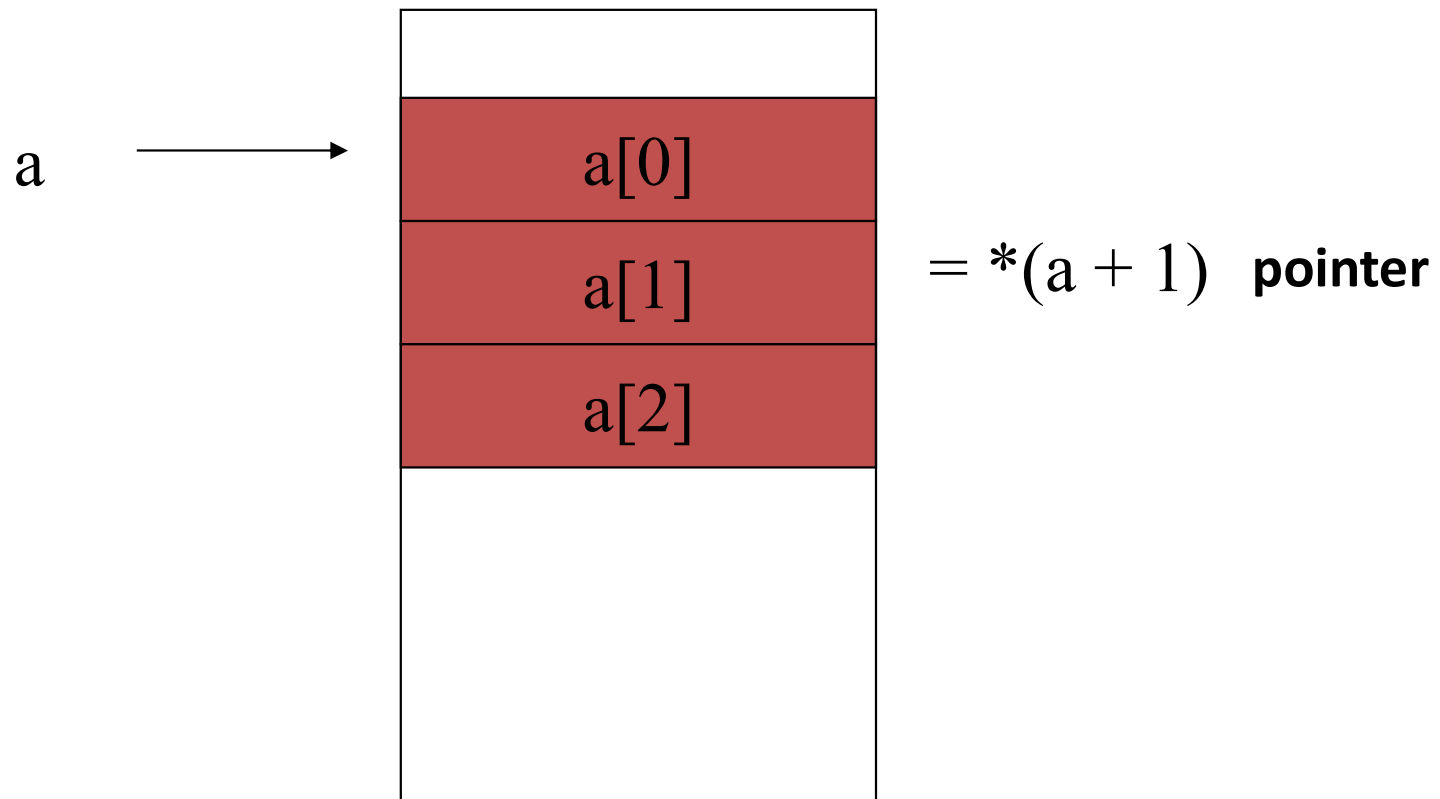
code generated by the ARM gcc compiler

Data structure transformations

- The compiler must also translate references to data structures into references to raw memories.
- Different types of data structures use different data layouts.
- Some offsets into data structure can be computed at compile time, others must be computed at run time.

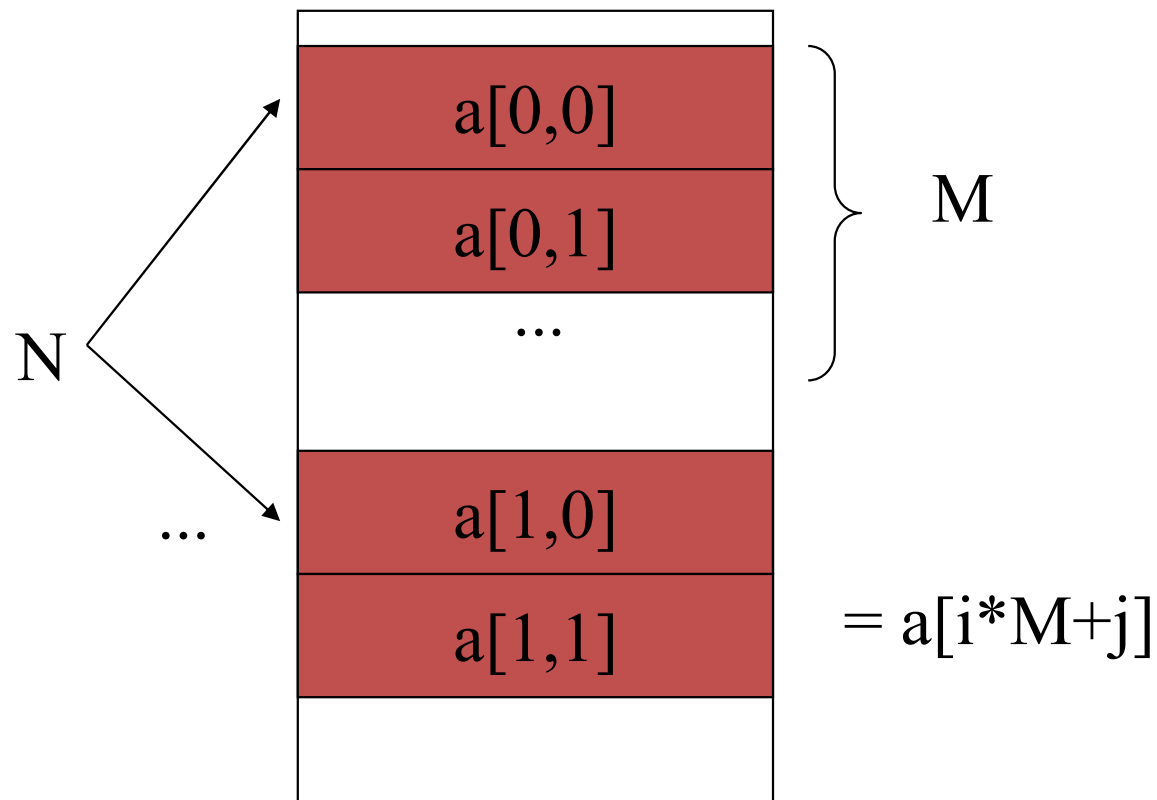
One-dimensional arrays

- C array name points to 0th element:



Two-dimensional arrays

- Column-major layout:



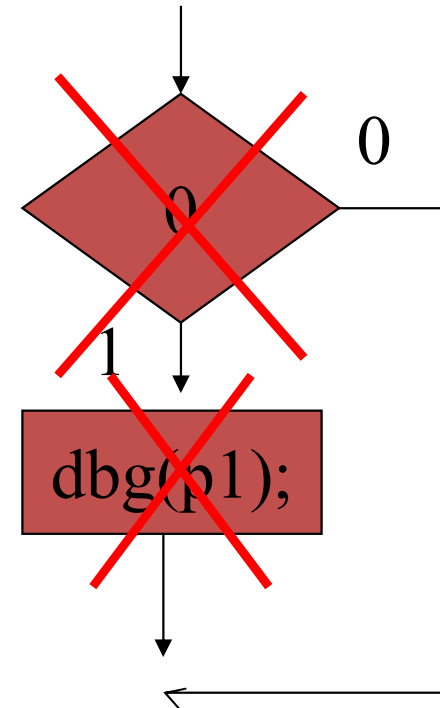
Compiler Optimizations

Expression simplification

- Constant folding:
 - $N+1 = 8+1 = 9$ (N has been declared as a constant)
- Algebraic:
 - $a*b + a*c = a*(b+c)$ (Why?)
- Strength reduction:
 - $a*2 = a \ll 1$

Dead code elimination

- Dead code:
`#define DEBUG 0`
`if (DEBUG) dbg(p1);`
- Can be eliminated by
analysis of control flow,
constant folding.



Procedure inlining

- Eliminates procedure linkage overhead:

```
int foo(a,b,c) { return a + b + c;}
```

```
z = foo(w,x,y);
```



```
z = w + x + y;
```

Register allocation

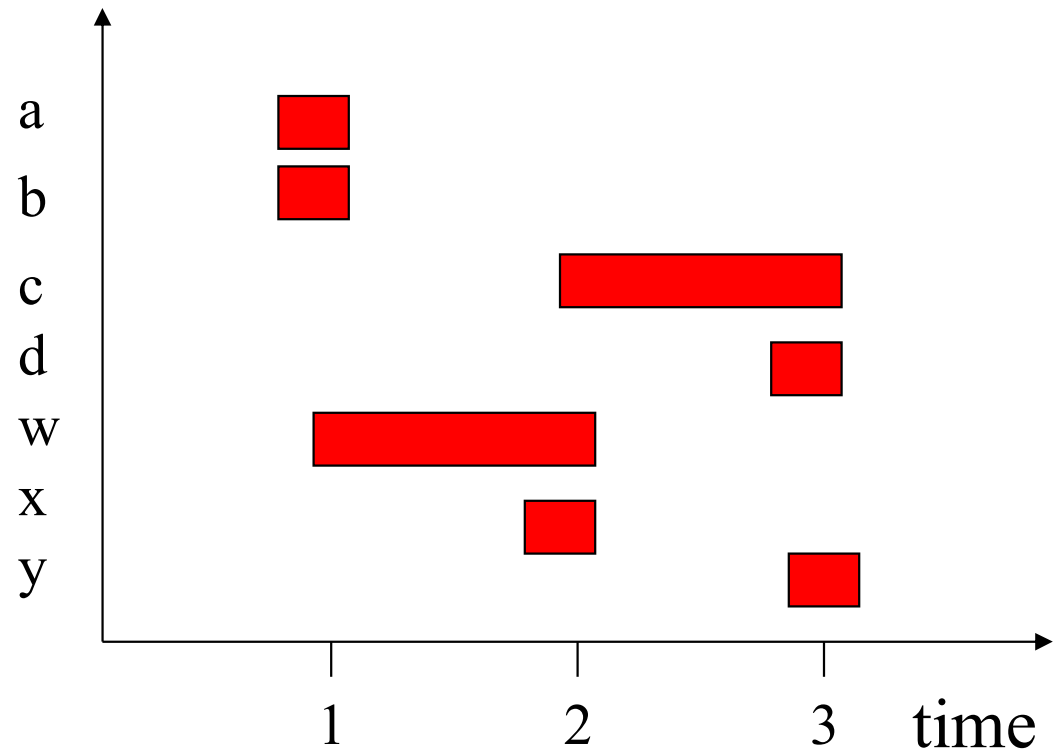
- Goals:
 - choose register to hold each variable;
 - determine lifespan of variable in the register.
- Basic case: within basic block.

Register lifetime graph

$w = a + b;$ $t=1$

$x = c + w;$ $t=2$

$y = c + d;$ $t=3$



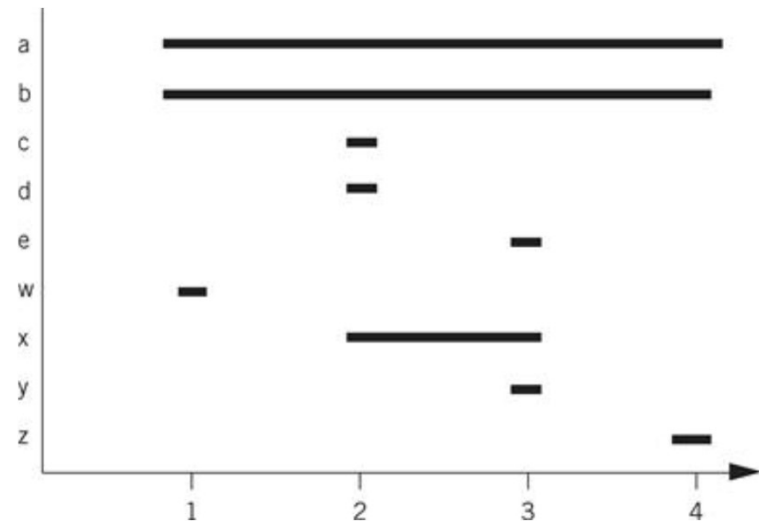
QUIZ



How to solve the situation where a section of code requires more registers than are available?

```
w = a + b; /* statement 1 */  
x = c + d; /* statement 2 */  
y = x + w; /* statement 3 */  
z = a - b; /* statement 4 */
```

Original code



Lifetime Graph

QUIZ



How to solve the situation where a section of code requires more registers than are available?

```
w = a + b; /* statement 1 */  
x = c + d; /* statement 2 */  
y = x + w; /* statement 3 */  
z = a - b; /* statement 4 */
```

Original code

```
w = a + b; /* statement 1 */  
z = a - b; /* statement 4 */  
x = c + d; /* statement 2 */  
y = x + w; /* statement 3 */
```

Modified code

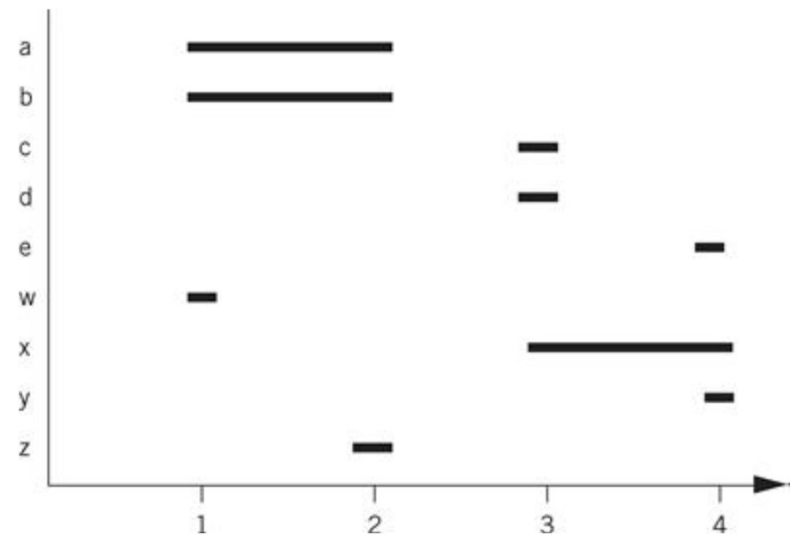
QUIZ



How to solve the situation where a section of code requires more registers than are available?

```
w = a + b; /* statement 1 */  
z = a - b; /* statement 4 */  
x = c + d; /* statement 2 */  
y = x + w; /* statement 3 */
```

Modified code



Lifetime Graph

Instruction scheduling

- When an instruction is executed and which resources does it use?
- In pipelined machines, execution time of one instruction depends on the nearby instructions: **opcode**, **operands**.

Summary

Quiz

Q1: **State machine example, circular buffer, Queues?**

Q2: For each basic block given below, rewrite it in single-assignment form, and then draw the data flow graph for that form.

a). $x = a + b;$
 $y = c + d;$
 $z = x + e;$

c). $a = q - r;$
 $b = a + t;$
 $a = r + s;$
 $c = t - u;$

b). $r = a + b - c;$
 $s = 2 * r;$
 $t = b - d;$
 $r = d + e;$

d). $w = a - b + c;$
 $x = w - d;$
 $y = x - 2;$
 $w = a + b - c;$
 $z = y + d;$
 $y = b * c;$

Quiz

Q3: Draw the CDFG for the following code fragments:

a).

```
if (y == 2) {r = a + b; s = c - d;}  
else r = a - c
```

b).

```
x = 1;  
if (y == 2) { r = a + b; s = c - d; }  
else { r = a - c; }
```

c).

```
x = 2;  
while (x < 40) {  
    x = foo[x];  
}
```

d).

```
for (i = 0; i < N; i++)  
    x[i] = a[i]*b[i];
```

e).

```
for (i = 0; i < N; i++) {  
    if (a[i] == 0)  
        x[i] = 5;  
    else  
        x[i] = a[i]*b[i];  
}
```

Q4: What are **common components of a firmware**? How does a firmware **execute**?

Quiz

Q5: Show the contents of the assembler's symbol table at the end of code generation for each line of the following programs:

a.

```
    ORG 200
p1: ADR r4,a
    LDR r0,[r4]
    ADR r4,e
    LDR r1,[r4]
    ADD r0,r0,r1
    CMP r0,r1
    BNE q1
p2: ADR r4,e
```

c.

```
    ORG 200
S1: ADR r2,a
    LDR r0,[r2]
S2: ADR r2,b
    LDR r2,a
    ADD r1,r1,r2
```

b.

```
    ORG 100
p1: CMP r0,r1
    BEQ x1
p2: CMP r0,r2
    BEQ x2
p3: CMP r0,r3
    BEQ x3
```

Quiz

Q6: Draw the CDFG for the following C code before and after applying dead code elimination to the if statement:

```
#define DEBUG 0
proc1();
if (DEBUG) debug_stuff();
switch (foo) {
    case A: a_case();
    case B: b_case();
    default: default_case();
}
```

Q7: Unroll the loop below:

```
for (i = 0; i < 32; i++)
    x[i] = a[i] * c[i];
```

- a. two times
- b. three times

Quiz

Q8: Apply loop fusion or loop distribution to these code fragments as appropriate. Identify the technique you use and write the modified code.

a.

```
for (i=0; i<N; i++)  
    z[i] = a[i] + b[i];  
for (i=0; i<N; i++)  
    w[i] = a[i] - b[i];
```

b.

```
for (i=0; i<N; i++) {  
    x[i] = c[i]*d[i];  
    y[i] = x[i] * e[i];  
}
```

c.

```
for (i=0; i<N; i++) {  
    for (j=0; j<M; j++) {  
        c[i][j] = a[i][j] + b[i][j];  
        x[j] = x[j] * c[i][j];  
    }  
    y[i] = a[i] + x[j];  
}
```

Q9: What is software pipelining? Give an example about software pipelining.